ESR Loot Filter Guide

Oct 11, 2026 · @Bogdan

Where filters live

You pick the active filter and its level in the ESR launcher's Loot Filter tab; the plugin reloads it whenever the file or the choice changes.

- **Filters that ship with the mod:** `<game>\mods\<mod>\LootFilters\`
- **Your own and imported filters:** `<Saved Games>\Diablo II Resurrected\mods\<mod>\LootFilters\`
- **Next to a filter:** a `sounds` folder for its sound files and an `icons` folder for its minimap icons.
- **selection.json** in your own filters folder keeps the active filter, the level for each filter and whether sounds play. The launcher writes it.
- **In game:** the Loot Filter section in Controls holds the level key, which steps through the filter's levels. The console command `lootfilter` shows the state; `lootfilter reload` reloads.

A filter file

A filter is one JSON file with `//` comments allowed: a few settings, optional styles, then the rules.

```
{
  "formatVersion": 1,
  "title": "My Filter",
  "author": "Me",
  "filterLevels": [ { "id": 1, "name": "Leveling" }, { "id": 2, "name":
"Strict" } ],
  "defaultLevel": 1,
  "soundVolume": 80,
  "notifications": { "dropFormat": "ÿc4Drop: {qualityColor}{name}" },
  "qualityColors": { "magic": "ÿc2" },
  "styles": {
    "HighRune": { "labelBackground": [200, 120, 0], "sound": "item_rune",
"notify": true, "droplight": { "color": [255, 128, 0] } }
  },
  "rules": [
    { "comment": "High runes", "itemCodes": ["r30", "r31", "r32"],
"useStyle": "HighRune" },
    { "itemTypes": ["pots"], "hide": true, "activeInLevels": [2] }
  ]
}
```

Key	What it does
formatVersion	Always 1.
title, author, version, description	Shown in the launcher.
updateUrl	The address the filter's newer copies are published at.
filterLevels, defaultLevel	Strictness levels, each `{ "id", "name" }`, and the one a new player starts on.
hideUnmatchedItems	true hides every ground item no rule matched.
soundVolume	0 to 100, the volume for rules that set none.
notifications	The drop line and the level change line (see Chat notifications).
qualityColors	Name colors per item quality (see Quality colors).
styles	Named sets of actions that rules pull in with `useStyle`.

Key	What it does
rules	The rules, applied in order.

How rules match

Every rule whose conditions all hold applies, top to bottom, until one hides the item.

- **AND and OR:** all conditions of a rule must hold; values inside one list are alternatives (`"itemCodes": ["r30", "r31"]` matches either).
- **Later wins:** a later rule's background, border, name change, icon or droplight replaces an earlier one's. Chat lines and sounds add up.
- **hide stops:** a rule with `"hide": true` hides the ground label and makes the item impossible to pick up (gold excepted, since it is picked up automatically), and no rule after it runs. Show Hidden Items, or the game's own loot filter toggle off, shows hidden items again and lets them be picked up.
- **hideIfOtherConditionsFail:** when the codes, types and location match but another condition fails, the item is hidden. Good for "show this base only with sockets".
- **Where:** rules look at ground items unless `location` says otherwise.
- **Levels:** `activeInLevels` limits a rule to some filter levels; without it the rule is active at every level.
- **Styles:** `useStyle` pulls in a style's actions; the rule's own actions win over the style's. Styles hold actions only.
- **Mistakes:** a rule with an error is switched off and logged with its number and line; the rest of the filter keeps working.
- **show protects:** after a rule with `"show": true` matches, later rules cannot hide the item, by `hide`, `hideIfOtherConditionsFail` or `hideUnmatchedItems`. Their colors, names, sounds and chat lines still apply.
- **Unidentified items:** a rule that checks an item stat skips them entirely (see Stat values).

Conditions

Numbers take one value (`5`), a list (`[1, 2]`) or a range (`{ "min": 3, "max": 5 }`); named values take the name or its number.

Key	Value	Holds when
itemCodes, excludeItemCodes	item codes, e.g. `["r01", "hax"]`	the item's code is (not) listed
itemTypes, excludeItemTypes	itemtypes.txt codes, e.g. `["rune", "weap"]`	the item is (not) of that type, parent types included
quality	inferior, normal, superior, magic, set, rare, unique, crafted, tempered (1 to 9)	the item's quality matches
baseTier	normal, exceptional, elite (0 to 2)	the base tier matches
itemLevel	0 to 999	the item level matches
sockets, maxSockets	0 to 99	sockets it has, or the most its base can have
isFullySocketed, isEthereal, isIdentified, isRuneword	true or false	the item is (not) that
uniqueOrSetIds, excludeUniqueOrSetIds	row numbers in uniqueitems.txt or setitems.txt	the unique or set item is (not) listed
itemStats, itemStatsAnyOf	stat checks	all of them (any of them) hold for the item
playerStats, playerStatsAnyOf	stat checks	all of them (any of them) hold for your character
location	onGround, stored, equipped, atVendor	the item is there; default onGround
difficulty	normal, nightmare, hell (0 to 2)	the game's difficulty matches
areas, excludeAreas	levels.txt Id numbers	you are (not) in one of those areas
activeInLevels	filter level ids	the filter is at one of those levels

Key	Value	Holds when
formula	a formula, see Formulas	the formula is not 0

A stat check is `{ "stat": "item_numsockets", "param": 0, "op": ">=", "value": 2 }`. `stat` is an itemstatcost.txt name or id, `param` is optional, `op` is `==`, `!=`, `>`, `>=` (the default), `<`, `<=` or `between` with `"value": [low, high]`.

Formulas

`formula` is a condition that holds when its formula is not 0. It is written like a txt calc and checked when the filter loads.

```
{ "itemTypes": ["ring"], "quality": "magic",
  "formula": "((stat('item_fastercastrate')>=10)&&
((stat('fireresist')+stat('coldresist')+stat('lightresist'))>=40))",
  "hideIfOtherConditionsFail": true }
```

Part	What it gives
`stat('name')`	The item's stat. `name` is an itemstatcost.txt Stat, or its row number.
`stat('name', param)`	One param of it: a number, or a skills.txt skill name such as `'Teleport'`.
`playerStat('name')`, `playerStat('name', param)`	Your character's stat.
`statMax('name')`, `statSum('name')`	The highest value, or the total, over every param of the item's stat.
`+ - * /`	Whole numbers; division rounds toward 0, and division by 0 gives 0.
`== != < <= > >=`	Comparisons give 1 or 0.
`&& || !`	And, or, not.
`a ? b : c`	b when a is not 0, else c.

- **Skill names on chance-to-cast and charges:** these stats keep the skill and its level in the param, so a skill name matches that skill at any level, e.g. `stat('item_charged_skill', 'Teleport')` gives the level of its Teleport charges.
- **Names in single quotes:** they need no escaping inside the JSON.
- **Mistakes:** a bad name or bad syntax switches the rule off, and the log gives the rule number, line and character.

Stat values

Every stat a filter reads is the number the item tooltip prints, in `itemStats`, `playerStats`, formulas and the `{stat:}` placeholders.

- **Life, mana and stamina:** stored in 256ths, so +40 life reads 40.
- **Per-level stats:** the value at your character's level, as "(Based on Character Level)" shows it. 0.5 per level reads 0 at level 1 and 49 at level 99.
- **item_howl and item_fear:** stored in 128ths; they read the percent.
- **Charges:** the skill level.
- **Everything else reads as stored,** including the stats whose line only flips the sign, item_replenish_durability and poison damage.
- **Unidentified items:** the client gets no modifiers until an item is identified, so a rule that checks any item stat skips an unidentified item: none of its actions run and it cannot hide it. gold, armorclass, quantity, durability, maxdurability and item_numsockets still work, because every item carries them. Items that drop identified, like quality items, work as usual.

Actions

Key	Value	Effect
hide	true	Hides the ground label, and while hidden the item cannot be hovered, targeted or picked up, with the mouse or a controller; gold stays pickable. No later rule runs.
hideIfOtherConditionsFail	true	See How rules match.

Key	Value	Effect
replaceName, namePrefix, nameSuffix	text	A new name, or text before or after it. `{name}` is the game's name.
labelBackground	`[r, g, b]` or `[r, g, b, a]`, 0 to 255	Fills the ground label; alpha defaults to 242.
labelBorder	`{ "color": [r, g, b], "width": 2 }`	Outlines the ground label.
chatMessage	text	A chat line when the item drops.
chatMessageWhenHidden	text	A chat line when this rule hides the item.
notify	true	Prints the filter's drop line (see Chat notifications).
sound	sounds.txt name or row, or a file in `sounds`	Plays when the item drops.
soundVolume	0 to 100	This rule's sound volume.
icon	`{ "iconName", "iconSize", "color" }`	A minimap icon (see Minimap icons).
droplight	`{ "color": [r, g, b] }`, or `{}`	The droplight effect on the item (see Droplight).
useStyle	a style's name	Pulls in that style's actions.
comment	text	Ignored; notes for the author.
show	true	Keeps the item visible: no later rule can hide it, and their other actions still apply. A rule cannot both show and hide.

Texts, placeholders and translations

Any text can be one string or one string per language, and `{name}` is always the game's own translated name.

- **Per language:** `{ "enUS": "Rune", "deDE": "Rune" }`. A missing language falls back to enUS. Languages: enUS, deDE, esES, frFR, itIT, koKR, plPL, ruRU, zhCN, zhTW, esMX, jaJP, ptBR.
- **Game strings:** `{string:KEY}` shows a key from the game's string files in the player's language, e.g. `{string:r01}` for El Rune.
- **Colors:** `ÿc` plus one character (see Color codes). In JSON `\u00ffc` works too. D2RLAN tags like `{red}` are not supported.

Placeholder	Shows
{name}	The item's name, translated by the game.
{code}, {quality}, {baseTier}, {itemLevel}	The item's code, quality, base tier and item level.
{sockets}, {maxSockets}, {ethereal}, {uniqueOrSetId}	Its sockets, socket limit, ethereal flag and unique or set row.
{qualityColor}	The color code of the item's name, after qualityColors.
{stat:ID} or {stat:ID,PARAM}	One of the item's stats.
{playerStat:ID} or {playerStat:ID,PARAM}	One of your character's stats.
{string:KEY}	A game string in the player's language.
{link}	Chat lines only: the item's name.
{level}, {filter}	levelChangeFormat only: the level's name and the filter's title.

Chat notifications

A rule with `"notify": true` prints the filter's drop line every time the item drops or comes into view; a rule's own `chatMessage` replaces that line.

- **dropFormat:** the drop line. Default `{qualityColor}{name}`.

- **levelChangeFormat:** printed when the level changes in game, by the key or in the launcher. Takes `{level}` and `{filter}`. Default `ÿc4Loot filter level: ÿc0{level}`.
- Both take per-language texts and `{string:KEY}`.
- **Off switch:** `"chatNotifications": false` in selection.json turns off every chat line about an item: drop lines, `chatMessage` and `chatMessageWhenHidden`. The filter announce, level changes and Show Hidden Items still print.

```
"notifications": {
  "dropFormat": "ÿc4Drop: {qualityColor}{name}",
  "levelChangeFormat": { "enUS": "Filter: {level}", "deDE": "Filter: {level}"
}
}
```

Sounds and volume

A sound plays every time its item drops; its volume is the rule's `soundVolume`, else the filter's, else 100.

- **sounds.txt sounds:** the Sound name (`"item_rune"`) or the row number. They also follow the game's sound volume.
- **Sound files:** a `.wav`, `.mp3` or `.flac` name in the filter's `sounds` folder. They are also scaled by `file_sound_volume` in the plugin's TOML.
- **Off switch:** the launcher's "Play sounds" turns every filter sound off.

Minimap icons

An `icon` draws a picture on the minimap where the item lies, for as long as it is on the ground and not hidden.

```
"icon": { "iconName": "rune", "iconSize": 16, "color": [255, 128, 0] }
```

Key	Value
iconName	A `.sprite` file (D2R's HD sprite format) in the filter's `icons` folder; `.sprite` may be left off. Only its first frame is used.

Key	Value
iconSize	4 to 256: the longer side in minimap pixels, keeping the file's proportions. Leave it out for the file's own size.
color	Optional tint `[r, g, b]`; a fourth number fades the icon.

Icons need HD graphics. If another plugin already hooks the minimap's unit drawing (Ruff's MapSense does), icons stay off and the plugin log says so.

Droplight

A `droplight` puts the droplight effect, a beam of light with a sparkle, on the item while it lies on the ground and is not hidden.

```
"droplight": { "color": [255, 128, 0] }
```

Key	Value
color	Optional `[r, g, b]`, 0 to 255: the beam's color. Leave it out, as in `"droplight": {}`, for the game's white beam.

- **HD only:** the droplight is an HD effect; legacy graphics show nothing.
- **The mod's part:** the plugin attaches the effect as the overlay.txt row named by `droplight_overlay` in its TOML (default `droplight`). Without that row droplights stay off and the plugin log says so.
- **Colors:** the plugin recolors a copy of the game's beam, `fx_horadric_light.particles`, which the mod ships at the path `droplight_beam` names. Without that file a colored droplight keeps the white beam and the plugin log says so. `droplight_brightness` in the plugin's TOML (1 to 100, default 50) sets how bright colored beams are, as a share of the game's white beam; near full brightness the beam's core clips and the color washes out.

Quality colors

`qualityColors` recolors item names by quality everywhere the game shows them: ground labels, the Alt and nearby lists, tooltips, the set item tooltip and `{qualityColor}`.

```
"qualityColors": { "magic": "ÿc2", "rare": "ÿc3", "unique": "ÿc;" }
```

- **Keys:** inferior, normal, superior, magic, set, rare, unique, crafted, tempered. **Values:** one color code.
- Only a quality's plain color changes. Runes, quest items, socketed or ethereal gray and broken red keep their own colors.

Color codes

A color code is `ÿc` plus one character, from `ÿc0` to `ÿcV` in ASCII order; the text after it keeps that color until the next code.

Code	Color	RGB	The game uses it for
ÿc0	white	255, 255, 255	plain text, normal items
ÿc1	red	255, 77, 77	broken items, warnings
ÿc2	green	0, 255, 0	set items (tooltips)
ÿc3	blue	110, 110, 255	magic items (tooltips)
ÿc4	gold	199, 179, 119	unique items (tooltips)
ÿc5	gray	105, 105, 105	socketed and ethereal items (tooltips)
ÿc6	black	0, 0, 0	
ÿc7	tan	208, 194, 125	
ÿc8	orange	255, 168, 0	crafted items (tooltips)
ÿc9	yellow	255, 255, 100	rare items (tooltips)
ÿc:	dark green	0, 128, 0	tempered items (tooltips)
ÿc;	purple	174, 0, 255	
ÿc<	green	0, 200, 0	

Code	Color	RGB	The game uses it for
ÿc=	white	240, 240, 240	plain item labels
ÿc>	gold	199, 179, 119	quest item labels
ÿc?	yellow	255, 255, 100	rare labels
ÿc@	orange	255, 168, 0	crafted labels
ÿcA	dark green	0, 128, 0	tempered labels
ÿcB	blue	110, 110, 255	magic labels
ÿcC	green	0, 252, 0	set labels
ÿcD	gold	199, 179, 119	unique labels
ÿcE	white	240, 240, 240	health potion labels
ÿcF	white	240, 240, 240	mana potion labels
ÿcG	white	240, 240, 240	rejuvenation potion labels
ÿcH	white	240, 240, 240	gold labels
ÿcI	gray	99, 99, 99	ethereal labels
ÿcJ	orange	255, 168, 0	rune labels
ÿcK	gray	99, 99, 99	socketed labels
ÿcL	orange	255, 168, 0	event item labels
ÿcM	light purple	192, 128, 242	desecrated labels
ÿcN	turquoise	6, 165, 221	
ÿcO	pink	255, 128, 255	
ÿcP	light blue	170, 170, 255	
ÿcQ	green	0, 255, 0	
ÿcR	yellow	255, 255, 118	
ÿcS	dark red	208, 62, 62	
ÿcT	sky blue	130, 197, 255	
ÿcU	red	255, 0, 0	

Code	Color	RGB	The game uses it for
ÿcV	red	255, 0, 0	

- **0 to <:** the `baseColors` list in `globaldatahd.json`, its 13 entries in order. They match the legacy palette except `ÿc3`, which HD draws as 110, 110, 255.
- **= to M:** the label colors of `TooltipStyle` in `_profilehd.json`: = Default, > Quest, ? Rare, @ Crafted, A Tempered, B Magic, C Set, D Unique, E HealthPotion, F ManaPotion, G RejuvPotion, H Gold, I Ethereal, J Rune, K Socketed, L EventItems, M Desecrated. Changing an entry there changes its code.
- **N to V:** chat colors fixed in the game exe. ESR's `color-codes-2x-letters` patch keeps their 2.x letters, so the light gold (238, 214, 141) that stock 3.3 puts on N is not reachable in ESR. Stock 3.3 shifts them one letter: N light gold, O turquoise, P pink, Q light blue, R green, S yellow, T dark red, U sky blue, V red.

Finding problems

Every error and warning goes to the D2RLoader plugin log with the rule number and line, and a rule with an error is switched off.

- **Console:** `lootfilter` shows the plugin state, which hooks are on, the active filter, its rule counts, level and problem counts. `lootfilter reload` reloads the filter.
- **`debug_log = true`** in the plugin's TOML logs one line per item decision (shown, hidden and by which rule), every chat line printed, every icon sprite loaded and every droplight added.
- **Missing files:** a sound or icon the filter names but its folder lacks is logged; that rule keeps working without it.
- **Droplights off:** the log names the reason, such as overlay.txt having no row named in `droplight_overlay`.

How to use filters for other mods

The plugin is not tied to ESR. It works for whichever mod the game runs: it finds that mod's folders itself and resolves every name in a filter against that mod's own files. Another mod can ship it with filters of its own.

What players install

- **D2RLoader and the plugin:** `d2rl-celestialrayone-loot-filter.dll` goes in `d2rloader\plugins\` and its settings file, `celestialrayone.loot-filter.toml`, in `d2rloader\config\`. Both folders sit either in the game folder (the plugin then runs for every mod) or in `<game>\mods\<mod>\` (only for that mod). The plugin writes its log to `d2rloader\logs\celestialrayone.loot-filter.log`.
- **The D2RLoader version the plugin is built for:** on another version, the plugin checks the game's code before each feature and switches off the ones that do not match. The log names them; everything else keeps working.
- **HD graphics** for minimap icons and droplights. Labels, chat lines, sounds and quality colors work in both modes.

Where the mod's files go

- **Filters the mod ships:** `<game>\mods\<mod>\LootFilters\`, one `.json` file per filter. `<mod>` is the mod's folder name, the one the game is started with (`-mod <mod>`).
- **Beside them:** a `sounds` folder for the filters' sound files and an `icons` folder for their minimap icon sprites.
- **Players' own filters:** `<Saved Games>\Diablo II Resurrected\mods\<mod>\LootFilters\`, or the folder `user_folder` in the plugin's TOML names. The plugin keeps `selection.json` there.

Picking the filter

- **Without a launcher:** set `default_filter` in the plugin's TOML to one of the mod's filter files, extension included, for example `default_filter = "my-filter.json"`. It is used until a player picks another one.
- **Levels:** players bind the keys under Options > Controls > Loot Filter; there are no default keys. Next Filter Level steps through the filter's levels, and Show Hidden Items shows hidden labels until it is pressed again. The plugin saves the level in `selection.json` itself.
- **With a launcher of the mod's own:** write `selection.json` in the players' filters folder. The plugin notices the change and applies it at once, even in game.

```
{
  "formatVersion": 1,
  "activeFilter": "shipped:my-filter.json",
  "levels": { "shipped:my-filter.json": 2 },
  "playSounds": true
}
```

Key	Value
activeFilter	`shipped:<file>` for a filter in the mod's folder, `user:<file>` for one in the players' folder, or `none` for no filter.
levels	The level each filter is at, by the same id: one of the `id` numbers of its `filterLevels`. A filter left out starts at its `defaultLevel`.
playSounds	`false` silences every filter sound.
chatNotifications	false turns off every chat line about an item. Missing means on.

- **Console:** `lootfilter` shows the active filter, its level and its problems; `lootfilter reload` reads the selection and the filter again.

Writing filters for the mod

- **Every name comes from the mod:** item codes from its weapons.txt, armor.txt and misc.txt, item types from its itemtypes.txt (parent types included), stats from itemstatcost.txt by name or id, sounds by their sounds.txt name, areas by their levels.txt Id, and unique or set items by their row in uniqueitems.txt or setitems.txt. ESR's filters use ESR's items, so they do not fit another mod as they are.
- **Names the mod lacks** are reported in the plugin log with the rule's number and line. A rule with an error is switched off; the rest of the filter keeps working.
- **Color codes:** `ÿcN` to `ÿcV` follow ESR's `color-codes-2x-letters` patch. A mod without that patch has the stock 3.3 letters, listed under Color codes.
- **Everything else in this guide applies as written:** conditions, actions, texts and translations, chat notifications, sounds, minimap icons and quality colors.

Droplights in another mod

- **overlay.txt row:** add one row named `droplight`, or the name `droplight_overlay` in the plugin's TOML sets. The plugin only uses the row's number and its legacy graphics are never drawn in HD, so a renamed copy of an existing looping overlay row, such as an aura's, is enough. Without the row, droplights stay off and the log says so.
- **Colored beams:** copy the game's own `data/hd/vfx/particles/overlays/object/horadric_light/fx_horadric_light.particles`, extracted from the game's storage with a CASC tool, into the mod at the same path inside its data folder (`<game>\mods\<mod>\<mod>.mpq\data\…`). The plugin makes a recolored copy of it for each color a filter uses. The file is the game's own, unchanged, so the game's horadric light looks the same. Without it, colored droplights keep the white beam.
- **Settings:** `droplight_beam` in the TOML changes where the plugin looks for that file, and `droplight_brightness` (1 to 100, default 50) sets how bright colored beams are.

Checking it works

- **In game:** the `lootfilter` console command lists which features are on, the active filter, its rule counts, its level and its problems.
- **The plugin log** has every error and warning with the rule number and line, and the reason when a feature is off.
- **`debug_log = true`** in the TOML also logs every item decision, chat line, icon sprite and droplight.